Aim: Program to implement Naïve Bayes Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm

Short notes: Naive Bayes

Bayes' Theorem provides a way that we can calculate the probability of a piece of data belonging to a given class, given our prior knowledge. Bayes' Theorem is stated as:

P(class|data) = (P(data|class) * P(class)) / P(data)

Where P(class|data) is the probability of class given the provided data.

We are using Iris Dataset. The Iris Flower Dataset involves predicting the flower species given measurements of iris flowers.

It is a multiclass classification problem. The number of observations for each class is balanced. There are 150 observations with 4 input variables and 1 output variable. The variable names are as follows:

Sepal length in cm.

Sepal width in cm.

Petal length in cm.

Petal width in cm., and

Class.

Algorithm:

Step 1: Separate By Class.

Step 2: Summarize Dataset.

Step 3: Summarize Data By Class.

Step 4: Gaussian Probability Density Function.

Step 5: Class Probabilities.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn import neighbors, datasets, preprocessing
from sklearn.model_selection import train_test_split

iris = datasets.load_iris()
X, y = iris.data[:, :], iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify =
y, random_state = 0, train_size = 0.7)
```

```
#Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In this step, we introduce the class GaussianNB that is used from the sklearn.naive_bayes library. Here, we have used a Gaussian model, there are several other models such as Bernoulli, Categorical and Multinomial. Here, we assign the GaussianNB class to the variable classifier and fit the X_train and y_train values to it for training purpose.

```
from sklearn.naive_bayes import GaussianNB, BernoulliNB, CategoricalNB
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
scores = []
classifier = GaussianNB()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
scores.append(accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[15,  0,  0],
       [ 0, 15,  0],
       [ 0,  1, 14]])
```

```
classifier1 = BernoulliNB()
classifier1.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
scores.append(accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[15,  0,  0],
       [ 0, 15,  0],
       [ 0,  1, 14]])
```

From the above confusion matrix, we infer that, out of 45 test set data, 44 were correctly classified and only 1 was incorrectly classified. This gives us a high accuracy of 97.7%.

```
#classifier1 = CategoricalNB()
#classifier1.fit(X_train, y_train)
#y_pred = classifier.predict(X_test)
#scores.append(accuracy_score(y_test, y_pred))
#ValueError: Negative values in data passed to CategoricalNB (input X)
```

```
print(scores)
```

```
[0.9777777777777777, 0.9777777777777777]
```